RAG Mastery

Retrieval-Augmented Generation from zero to production

chunking, embeddings, vector DBs, hybrid retrieval, reranking, query transforms, grounded generation

evaluation with RAGAS

advanced + agentic RAG

the comparisons interviewers love

model answers

rapid-fire Q&A

glossary

@rathoreadiitya

@rathoreadiitya

v1.0 · Updated 2026-06-27

*New to RAG? Start at **§2 Why LLMs need RAG**, then read top to bottom. Prepping for an interview? Jump to Part 2 (§17).*

RAG is the most common way real companies put an LLM to work on their own data: support docs, contracts, codebases, product catalogues. Instead of retraining a model, you fetch the right text at question time and hand it to the model as context. This sheet has two halves. Part 1 builds the pipeline from scratch: why LLMs hallucinate, what retrieve-then-generate means, then every stage in order (loading, chunking, embeddings, vector databases, retrieval, reranking, query rewriting, and grounded generation) with runnable LangChain and LlamaIndex code. Part 2 is the part that gets you hired and keeps you in production: how to evaluate a RAG system, the advanced patterns (contextual retrieval, GraphRAG, agentic RAG), the cost/latency/security tradeoffs, the comparison tables you get quizzed on, full model answers, a rapid-fire bank, and a glossary.

The student minimum. If you only revise five things the night before, revise these: (1) RAG = retrieve relevant chunks, then generate an answer grounded in them (§3); (2) embeddings turn text into vectors so similar meaning sits close together, and cosine similarity measures closeness (§9); (3) chunking decides everything downstream - too big and retrieval is noisy, too small and you lose context (§7); (4) hybrid retrieval (semantic + keyword) beats either one alone, and a reranker on top fixes the rest (§12, §13); (5) you cannot improve what you do not measure - retrieval recall and faithfulness are the two numbers that matter (§17).

Inside this sheet

PART 1 - RAG FROM SCRATCH

1. Who this sheet is for
2. Why LLMs need RAG - the four limits
3. What RAG is - the open-book exam
4. The RAG pipeline at a glance
5. RAG vs fine-tuning vs long-context vs agents
6. Ingestion 1 - loading documents
7. Ingestion 2 - chunking, the make-or-break step
8. Ingestion 3 - metadata that powers filtering
9. Embeddings - turning meaning into vectors
10. Vector databases - storing and searching vectors
11. Indexing - putting ingestion together
12. Retrieval - semantic vs keyword vs hybrid
13. Reranking and diversity - top-k done right
14. Query transformations - fixing the question
15. Generation - grounding, citations, refusing
16. End-to-end RAG in LangChain and LlamaIndex

PART 2 - PRODUCTION AND MASTERY

@rathoreadiitya

17. Evaluation - the numbers that matter
18. Advanced RAG patterns
19. Production - latency, cost, freshness, security
20. Failure modes and how to fix them
21. The comparisons interviewers love
22. Model answers - say these out loud
23. Rapid-fire bank
24. A 10-day study plan
25. Glossary
26. Final word

1. Who this sheet is for

If you have called an LLM API a few times but freeze when someone asks how you would make it answer from your company's PDFs, why your bot keeps making things up, or how you would know if your retrieval is any good, this is for you. No fluff. Each stage gets the plain-words idea, real Python, the memory hook, and the exact trap that trips people up.

2. Why LLMs need RAG - the four limits

An LLM is a very good text predictor trained on a frozen snapshot of the internet. That single fact creates four problems that RAG exists to fix.

Limit	What goes wrong	How RAG fixes it
Knowledge cutoff	the model knows nothing after its training date, so it cannot answer about last week's release	fetch fresh documents at question time
No private data	it never saw your internal wiki, contracts, or codebase, so it guesses	retrieve from your own indexed corpus
Hallucination	when unsure it produces fluent, confident, wrong answers	force it to answer from supplied text, not memory
No citations	you cannot tell where an answer came from or trust it	every answer points back to the source chunk

Yaad rakho - LLM ek brilliant student hai jisne saari purani kitaabein ratt li hain, par exam hall mein naya syllabus nahi pata aur tumhari company ki notes kabhi dekhi hi nahi. RAG usko sahi page khol ke de deta hai.

Trap: RAG does not make the model smarter, it makes it better-informed. If the right answer is not in the documents you retrieve, RAG cannot save you. Garbage retrieval gives garbage answers, no matter how good the model is.

@rathoreadiitya

3. What RAG is - the open-book exam

RAG stands for Retrieval-Augmented Generation. In one line: instead of asking the model to answer from memory, you first retrieve the most relevant pieces of text from your own data, then ask the model to answer using only those pieces.

The exam analogy that makes it click

- Closed-book exam = a plain LLM call. It answers from whatever it memorised. Confident, but it will bluff when it does not know.
- Open-book exam = RAG. Before answering, it flips to the right pages of the textbook (retrieval) and writes the answer from those pages (generation).
- Your job as the engineer is to make sure the right pages are easy to find and that the model is told to only use them.

The three letters

Letter	Stage	Plain meaning
R	Retrieval	find the chunks of text most relevant to the question
A	Augmented	stuff those chunks into the prompt as context
G	Generation	the LLM writes the final answer from that context

Yaad rakho - R-A-G ko aise yaad karo: Retrieve karo (sahi page dhoondo), Augment karo (woh page prompt mein chipka do), Generate karo (model jawab likhe usi page se). Teeno step alag hain - har step alag se sudhar sakte ho.

4. The RAG pipeline at a glance

A RAG system has two phases. The offline phase (indexing) you run once and refresh as data changes. The online phase (query) runs on every user question. Keep them separate in your head - most bugs live in one phase or the other.

Offline - indexing (build the library)

Indexing phase

```
documents  ->  load        # read PDFs, HTML, DB rows into text
           ->  chunk       # split into retrievable passages
           ->  embed       # each chunk -> a vector
           ->  store       # vectors + text + metadata -> vector DB
```

Online - query (answer a question)

Query phase

```
question   ->  embed         # same model as indexing
           ->  retrieve      # nearest chunks from the vector DB
           ->  (rerank)      # optional: reorder by true relevance
```

```
        -> augment       # build prompt = system + chunks + question
        -> generate      # LLM writes a grounded answer
        -> cite          # attach the sources it used
```

Golden rule: the query must be embedded with the EXACT same model you used to embed the documents. Mixing embedding models is like indexing a library in English and searching it in French - the coordinates do not line up.

Yaad rakho - Do phase yaad rakho: pehle library banao (load, chunk, embed, store), phir har sawaal pe library se page nikaalo (embed, retrieve, rerank, generate). Library ek baar banti hai, sawaal baar-baar aate hain.

5. RAG vs fine-tuning vs long-context vs agents

A favourite interview question is when NOT to use RAG. RAG is for injecting knowledge. It is not the only tool, and picking the wrong one is a classic mistake.

Approach	Best for	Weakness
RAG	answering from large, changing, private knowledge with citations	adds a retrieval step that can fetch the wrong thing
Fine-tuning	teaching a style, format, or skill (tone, JSON output, classification)	expensive to update, bad at fresh facts, can still hallucinate
Long-context (stuff it all in the prompt)	small, stable corpora that fit in the context window	cost and latency scale with tokens, accuracy drops in the middle
Agents / tools	tasks needing actions, calculations, or live APIs	more moving parts, harder to make reliable

The honest decision rule

- Need fresh or private FACTS, with sources? RAG.
- Need a new BEHAVIOUR or output format? Fine-tune (or just prompt well first).
- Corpus is tiny and never changes? Long-context may be simpler than RAG.
- It is not either/or - production systems often fine-tune for format AND use RAG for facts.

Trap: Do not fine-tune to add knowledge. Fine-tuning bakes in patterns, not facts, and you would have to retrain every time a document changes. Knowledge that changes belongs in retrieval, not in weights.

6. Ingestion 1 - loading documents

Step one of indexing is turning messy source files into clean text. Both LangChain and LlamaIndex ship dozens of loaders. The quality of this step is invisible but decides everything - bad extraction (broken tables, missing text from scanned PDFs) poisons the whole pipeline.

LangChain - loaders

```
from langchain_community.document_loaders import (
    PyPDFLoader, WebBaseLoader, TextLoader,
)

docs = PyPDFLoader("handbook.pdf").load()
# each Document has .page_content (text) and .metadata (source, page)
print(docs[0].page_content[:200])
print(docs[0].metadata)   # {'source': 'handbook.pdf', 'page': 0}
```

What to watch for

- Scanned PDFs are images - you need OCR (for example, unstructured or a vision model), a plain PDF loader returns nothing.
- Tables and multi-column layouts break naive extraction. Use a layout-aware parser (unstructured, LlamaParse) for those.
- Keep the source path and page number in metadata now - you will need them for citations later.

Yaad rakho - Loading ka rule simple hai: jo text saaf nahi nikla, woh kabhi retrieve nahi hoga. Pehle ek chunk khol ke aankhon se padho - agar tumhe samajh nahi aaya, model ko bhi nahi aayega.

7. Ingestion 2 - chunking, the make-or-break step

You cannot embed a whole 80-page PDF as one vector - it would average out into meaningless mush. So you split documents into chunks, and each chunk becomes one searchable unit. Chunking is the single most important decision in RAG. Get it wrong and nothing downstream can save you.

The size tradeoff

Chunk size	What happens	Symptom
Too small (a sentence)	each chunk lacks context; the answer is split across many chunks	retrieval misses, answers feel incomplete
Too big (a whole page)	one chunk mixes many topics; the embedding is blurry	retrieves noise, wastes context budget, weak similarity
Just right (a paragraph / idea)	one chunk = one coherent idea	clean retrieval, tight answers

The four chunking strategies, weakest to strongest

Strategy	How it splits	Use when

Strategy	How it splits	Use when
Fixed-size	every N characters/tokens, hard cut	quick baseline, uniform text
Recursive	tries paragraphs, then sentences, then words - keeps natural boundaries	the sensible default for prose
Document-structure	splits on headings, Markdown sections, code functions	structured docs, wikis, code
Semantic	splits where the topic actually shifts (embedding distance between sentences)	high-value corpora, when quality > speed

LangChain - recursive splitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,        # target size in characters
    chunk_overlap=120,     # carry context across the boundary
    separators=["\n\n", "\n", ". ", " ", ""],
)
chunks = splitter.split_documents(docs)
print(len(chunks), "chunks")
```

Why overlap matters

Overlap repeats a little text at the start of each chunk so an idea that straddles a boundary is not cut in half. A common starting point is a chunk of 500-1000 tokens with 10-20 percent overlap, then tune with evaluation. There is no universal best number - it depends on your documents and your questions.

Yaad rakho - Chunking ko roti ki tarah socho: bahut chhoti boti (sentence) se pet nahi bharta, poori roti (page) ek saath mooh mein nahi aati. Ek aache nivaale jitna - ek idea jitna - chunk perfect hai. Overlap matlab har boti thodi pichhli boti se judi rahe taaki baat toote na.

Trap: Do not pick a chunk size by gut and move on. Chunk size and overlap are hyperparameters - change them, then measure retrieval recall on a test set. Teams routinely double their accuracy just by retuning chunking.

8. Ingestion 3 - metadata that powers filtering

Every chunk should carry metadata: where it came from, when, what section, what type, who can see it. Metadata is not decoration - it powers filtered retrieval, citations, freshness, and access control.

Metadata + filtered retrieval

```
chunk.metadata = {
    "source": "handbook.pdf",
    "page": 12,
    "section": "Leave Policy",
```

```
    "updated": "2026-01-04",
    "team": "hr",          # for access control
}

# later, retrieve only fresh HR chunks the user is allowed to see
retriever = store.as_retriever(
    search_kwargs={"filter": {"team": "hr"}, "k": 5}
)
```

Yaad rakho - Metadata har chunk ka address aur ID card hai. Address se citation milta hai, ID card se decide hota hai kaun dekh sakta hai. Abhi mehnat karo, baad mein security aur sources free mein mil jaayenge.

9. Embeddings - turning meaning into vectors

An embedding model reads a chunk of text and returns a list of numbers (a vector) that captures its meaning. The trick: text with similar meaning gets vectors that point in similar directions, even if the words are different. "How do I reset my password" and "forgot login credentials" land close together.

How closeness is measured

- Each chunk becomes a point in a high-dimensional space (often 384 to 3072 numbers).
- Cosine similarity measures the angle between two vectors. Closer angle = more similar meaning. It runs from -1 (opposite) to 1 (identical).
- Retrieval is just: embed the question, find the chunk vectors with the highest cosine similarity.

Embedding a query

```
from langchain_openai import OpenAIEmbeddings

emb = OpenAIEmbeddings(model="text-embedding-3-small")  # 1536 dims
v = emb.embed_query("how do I reset my password")
print(len(v))   # 1536 numbers describing the meaning

# open-source, runs locally, free:
from langchain_huggingface import HuggingFaceEmbeddings
emb = HuggingFaceEmbeddings(model_name="all-MiniLM-L6-v2")  # 384 dims
```

Choosing an embedding model

Factor	What it means	Rule of thumb
Dimensions	length of the vector; more can capture more nuance	384-768 is plenty for most apps; bigger is not always better
Max sequence	longest text it can embed at once	must be >= your chunk size or the tail is silently dropped
Domain	general vs code vs legal vs multilingual	match the model to your text; check the MTEB leaderboard

Factor	What it means	Rule of thumb
Cost / hosting	API (paid, easy) vs local (free, you run it)	prototype on an API, move to local to cut cost at scale

Yaad rakho - Embedding matlab har sentence ko ek GPS coordinate de do. Matlab paas hai toh coordinate paas. Sawaal ka coordinate nikaalo, sabse paas waale chunks utha lo. Cosine similarity bas yeh naapta hai ki do points kitne paas hain.

Trap: The same model must embed both documents and queries, and you must re-embed your whole corpus if you ever switch models. The vectors from two different models are not comparable - their coordinate systems are unrelated.

10. Vector databases - storing and searching vectors

Once every chunk is a vector, you need somewhere to store millions of them and find the nearest ones to a query in milliseconds. Scanning every vector (brute force) is exact but too slow at scale, so vector databases use Approximate Nearest Neighbour (ANN) indexes - they trade a tiny bit of recall for a massive speed-up.

The key idea - ANN and HNSW

HNSW (Hierarchical Navigable Small World) is the most common ANN index. Picture a multi-layer graph: the top layer has a few long-range links to jump across the space fast, lower layers add local links to home in on the exact neighbourhood. A search hops down the layers, getting closer each time. The result is near-exact nearest neighbours, far faster than scanning everything.

Store	What it is	Reach for it when
FAISS	a library (in-memory/on-disk), not a server	prototyping, single-machine, full control, free
Chroma	embedded, dev-friendly local DB	quick local apps and notebooks
pgvector	a Postgres extension for vectors	you already run Postgres and want one system
Pinecone	fully managed cloud vector DB	you want zero ops and easy scaling, paid
Qdrant / Weaviate / Milvus	open-source vector DBs, self-host or cloud	production scale, rich filtering, hybrid search

LangChain - FAISS

```
from langchain_community.vectorstores import FAISS
from langchain_openai import OpenAIEmbeddings

emb = OpenAIEmbeddings(model="text-embedding-3-small")
store = FAISS.from_documents(chunks, emb)   # embeds + indexes
store.save_local("faiss_index")
```

```
# search: nearest 4 chunks to a query
hits = store.similarity_search("how much annual leave?", k=4)
```

Yaad rakho - Vector DB ek bahut bada library hai jahan kitaabein meaning ke hisaab se rakhi hain, alphabet se nahi. ANN/HNSW matlab librarian ko poori library nahi chhanni padti - woh seedha sahi shelf pe pahunch jaata hai.

11. Indexing - putting ingestion together

Here is the whole offline phase end to end - the same four steps in both frameworks. Run this once, then refresh when documents change.

LangChain - full indexing

```
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

docs   = PyPDFLoader("handbook.pdf").load()        # 1. load
chunks = RecursiveCharacterTextSplitter(
    chunk_size=800, chunk_overlap=120).split_documents(docs)  # 2. chunk
emb    = OpenAIEmbeddings(model="text-embedding-3-small")    # 3. embed
store  = FAISS.from_documents(chunks, emb)          # 4. store
store.save_local("faiss_index")
```

LlamaIndex - full indexing

```
from llama_index.core import SimpleDirectoryReader, VectorStoreIndex
from llama_index.core.node_parser import SentenceSplitter

docs  = SimpleDirectoryReader("./data").load_data()   # load
index = VectorStoreIndex.from_documents(               # chunk + embed + store
    docs,
    transformations=[SentenceSplitter(chunk_size=512, chunk_overlap=64)],
)
index.storage_context.persist("./storage")
```

12. Retrieval - semantic vs keyword vs hybrid

Retrieval is finding the chunks most relevant to a question. There are two families, and the best systems combine them.

Type	How it matches	Strong / weak at

Type	How it matches	Strong / weak at
Semantic (dense)	embedding similarity - matches meaning	great at paraphrases; weak at exact codes, names, IDs
Keyword (sparse / BM25)	term overlap - matches exact words	great at IDs, error codes, names; blind to synonyms
Hybrid	run both, blend the scores	best of both; the production default

Why hybrid wins: ask "what is the refund window for SKU-4471" and semantic search nails "refund window" but may miss the exact code SKU-4471, while BM25 nails the code but misses the paraphrase. Together they cover each other.

LangChain - hybrid retrieval

```
from langchain.retrievers import EnsembleRetriever
from langchain_community.retrievers import BM25Retriever

semantic = store.as_retriever(search_kwargs={"k": 5})
keyword  = BM25Retriever.from_documents(chunks); keyword.k = 5

hybrid = EnsembleRetriever(
    retrievers=[semantic, keyword],
    weights=[0.6, 0.4],          # tune on your eval set
)
hits = hybrid.invoke("refund window for SKU-4471")
```

Yaad rakho - Semantic search matlab meaning se dhoondo, keyword search matlab exact shabd se. Naam, code, ID dhoondhne ho toh keyword jeetta hai; matlab samajhna ho toh semantic. Hybrid dono ko mila deta hai - isliye production mein wahi default hai.

Trap: Pure semantic search quietly fails on exact tokens - part numbers, error codes, person names, acronyms. If users search by such tokens, you need keyword or hybrid, or you will look broken on the easy cases.

13. Reranking and diversity - top-k done right

First-stage retrieval is fast but rough - it uses a bi-encoder that embeds query and chunk separately. A reranker is a cross-encoder that reads the query and a candidate chunk together and scores true relevance. It is slower, so you only run it on the top results.

The retrieve-then-rerank pattern

- Stage 1 - retrieve a wide net cheaply (say top 25) with hybrid search.
- Stage 2 - rerank those 25 with a cross-encoder and keep the best 4-5.
- Result - you feed the model a small, high-precision context. Less noise, lower token cost, better answers.

LangChain - reranking

```
from langchain.retrievers import ContextualCompressionRetriever
from langchain_cohere import CohereRerank

base = store.as_retriever(search_kwargs={"k": 25})   # wide net
reranked = ContextualCompressionRetriever(
    base_compressor=CohereRerank(top_n=5),             # keep best 5
    base_retriever=base,
)
hits = reranked.invoke("how much annual leave do I get?")
```

MMR - kill duplicate chunks

Maximal Marginal Relevance picks chunks that are relevant AND different from each other, so you do not waste your context window on five copies of the same paragraph. Use it when your corpus has lots of near-duplicate text.

Yaad rakho - Retrieval ek jaali daal ke 25 machhli pakadta hai (tez par mota-mota). Reranker un 25 ko dhyaan se dekh ke top 5 chunta hai (slow par sahi). MMR yeh ensure karta hai ki 5 alag-alag hon, ek hi cheez ki copy nahi.

14. Query transformations - fixing the question

Users ask short, vague, or messy questions. You get better retrieval by rewriting the question before you search. These tricks live between the user and the retriever.

Technique	What it does	Helps when
Query rewriting	an LLM cleans a vague or chatty question into a crisp search query	conversational or typo-heavy input
Multi-query	generate several phrasings, retrieve for each, merge results	one phrasing misses relevant chunks
HyDE	ask the LLM to draft a hypothetical answer, embed THAT, search with it	the question and the answer use different words
Decomposition	split a complex question into sub-questions, retrieve for each	multi-part or comparison questions
Routing	send the query to the right index or tool	you have several corpora or data sources

LangChain - multi-query

```
from langchain.retrievers.multi_query import MultiQueryRetriever
from langchain_openai import ChatOpenAI

retriever = MultiQueryRetriever.from_llm(
    retriever=store.as_retriever(search_kwargs={"k": 4}),
    llm=ChatOpenAI(model="gpt-4o-mini", temperature=0),
)
# one user question -> several rephrasings -> merged, de-duped hits
hits = retriever.invoke("leave rules?")
```

Yaad rakho - Sawaal hi tedha ho toh jawab tedha aayega. Pehle sawaal ko seedha karo - saaf likho (rewriting), do-teen tareeke se poochho (multi-query), ya ek dummy jawab bana ke usse search karo (HyDE). Achha retrieval aadha kaam hai.

15. Generation - grounding, citations, refusing

Now the LLM writes the answer. The whole point of RAG is that it must answer from the retrieved chunks, not from memory. Your prompt is where you enforce that.

A grounded prompt template

Grounding prompt

```
SYSTEM = (
  "You answer ONLY using the context below. "
  "If the answer is not in the context, say you do not know. "
  "Cite the source after each claim like [source: <name> p<page>]."
)

prompt = f"""{SYSTEM}

Context:
{retrieved_chunks_with_metadata}

Question: {question}
Answer:"""
```

The three rules of grounded generation

- Tell the model to use ONLY the context. This is what cuts hallucination.
- Tell it to say "I do not know" when the context lacks the answer. A system that refuses cleanly is more trustworthy than one that bluffs.
- Make it cite the source chunk. Citations let users verify, and let YOU debug bad retrieval.

Context-window discipline

Do not dump 50 chunks into the prompt. More context is not better - models lose track of facts buried in the middle of a long prompt (the lost-in-the-middle effect), and every token costs money and latency. Retrieve wide, rerank, then pass a tight 3-6 chunks with the most important ones near the top.

Yaad rakho - Generation ka mantra: 'sirf isi context se jawab do, na mile toh saaf bolo pata nahi, aur har baat ke aage source likho.' Yeh teen line hallucination 80 percent kam kar deti hain.

Trap: If you do not explicitly forbid outside knowledge and allow "I do not know", the model

will happily fill gaps with confident fiction - and your citations will point at chunks that do not actually support the claim. Grounding is a prompt instruction, not automatic.

16. End-to-end RAG in LangChain and LlamaIndex

Here is a complete, minimal query pipeline in both frameworks. Indexing from §11 already built the store, so this is just the online phase.

LangChain - full RAG (LCEL)

```
from langchain_openai import ChatOpenAI, OpenAIEmbeddings
from langchain_community.vectorstores import FAISS
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser

emb   = OpenAIEmbeddings(model="text-embedding-3-small")
store = FAISS.load_local("faiss_index", emb,
                         allow_dangerous_deserialization=True)
retriever = store.as_retriever(search_kwargs={"k": 4})

prompt = ChatPromptTemplate.from_template(
    "Answer ONLY from the context. If missing, say you do not know.\n"
    "Context:\n{context}\n\nQuestion: {question}"
)

def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

chain = (
    {"context": retriever | format_docs,
     "question": RunnablePassthrough()}
    | prompt
    | ChatOpenAI(model="gpt-4o-mini", temperature=0)
    | StrOutputParser()
)
print(chain.invoke("how much annual leave do I get?"))
```

LlamaIndex - full RAG

```
from llama_index.core import StorageContext, load_index_from_storage

index  = load_index_from_storage(
    StorageContext.from_defaults(persist_dir="./storage"))
engine = index.as_query_engine(similarity_top_k=4)
```

```
resp   = engine.query("how much annual leave do I get?")
print(resp)                 # the answer
print(resp.source_nodes)    # the chunks it used (for citations)
```

Yaad rakho - Dekho pipeline ban gaya: retriever chunks laata hai, format_docs unhe ek text mein jodta hai, prompt context aur sawaal ko milata hai, model jawab deta hai. Har arrow ek alag, sudhaarne-yogya step hai.

17. Evaluation - the numbers that matter

You cannot improve a RAG system by vibes. The reason most demos never ship is that nobody measured them. RAG has two halves to evaluate separately: did retrieval fetch the right chunks, and did generation answer faithfully from them.

Retrieval metrics (is the right chunk in the results?)

Metric	Question it answers	Read it as
Recall@k	did the correct chunk appear in the top k?	the single most important RAG number
Precision@k	what fraction of the top k were relevant?	how much noise you are feeding the model
MRR	how high up was the first correct chunk?	higher = the right answer is near the top
nDCG	are the most relevant chunks ranked highest?	rank quality, rewards good ordering

Generation metrics with RAGAS (is the answer honest and useful?)

RAGAS metric	What it checks
Faithfulness	is every claim in the answer supported by the retrieved context? (catches hallucination)
Answer relevancy	does the answer actually address the question?
Context precision	are the retrieved chunks relevant and well-ranked?
Context recall	did retrieval fetch everything needed to answer fully?

RAGAS evaluation

```
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall,
)

# dataset rows: question, answer, contexts, ground_truth
report = evaluate(
    dataset,
    metrics=[faithfulness, answer_relevancy,
             context_precision, context_recall],
)
print(report)   # one score per metric, 0..1
```

Build a golden set

Write 30-50 real questions with known correct answers and the chunks that contain them. Run it after every change. This golden set is the difference between guessing and engineering - it tells you instantly whether a new chunk size, embedding model, or reranker actually helped.

Yaad rakho - Do cheezein alag-alag naapo: retrieval ne sahi chunk laaya (recall), aur model ne usi chunk se jawab diya (faithfulness). Agar jawab galat hai, pehle yeh dekho dosh kiska - dhoondhne ka ya likhne ka.

Trap: If you only look at the final answer, you cannot tell whether retrieval or generation failed. Always log the retrieved chunks too. A wrong answer from correct chunks is a prompt problem; a wrong answer from wrong chunks is a retrieval problem. Different fixes.

18. Advanced RAG patterns

Once the basic pipeline works, these patterns squeeze out more accuracy or handle harder questions. You do not need all of them - reach for the one that matches your failure mode.

Pattern	The idea	Use when
Parent-document	retrieve on small precise chunks, but feed the model the larger parent passage	small chunks retrieve well but lack context
Sentence-window	embed single sentences, return them with a window of neighbours	you need pinpoint matches plus surrounding context
Contextual retrieval	prepend a short LLM-written summary of the document to each chunk before embedding	chunks lose meaning without their document context (Anthropic, 2024)
Multi-hop	retrieve, reason, retrieve again using what you learned	answers need facts chained across documents
Self-RAG / CRAG	the model judges its own retrieval and re-retrieves or corrects if it is weak	you want the system to catch its own bad retrieval
Agentic RAG	an agent decides when to search, which tool to use, and when it has enough	open-ended tasks, multiple sources and tools
GraphRAG	build a knowledge graph of entities and relations, retrieve over it	global questions over a whole corpus (Microsoft GraphRAG)

Yaad rakho - Advanced patterns alag-alag bimaari ki dawa hain. Chunk mein context kam hai? Parent-document ya contextual retrieval. Sawaal multi-step hai? Multi-hop ya agentic. Poore corpus ka summary chahiye? GraphRAG. Pehle bimaari pakdo, phir dawa do.

Trap: Do not bolt on GraphRAG or agentic RAG before basic retrieval works and is measured. Most real wins come from better chunking, hybrid search, and a reranker - the boring stuff.

Reach for fancy patterns only when your golden set shows the simple pipeline has hit a wall.

19. Production - latency, cost, freshness, security

A notebook demo and a production RAG service are different animals. These are the concerns that decide whether it survives real users.

Concern	What bites you	What to do
Latency	retrieval + rerank + a long generation feels slow	cache, stream the answer, rerank fewer chunks, smaller models
Cost	embeddings + every query's context tokens add up	cache embeddings, trim context, use cheaper models for easy queries
Freshness	documents change but the index is stale	incremental re-indexing, upserts, store an updated timestamp
Caching	the same questions get asked again and again	cache embeddings and full answers (semantic cache) for repeat queries
Security	prompt injection, PII leakage, wrong-user access	filter by access metadata, strip/redact PII, treat retrieved text as untrusted
Scaling	millions of vectors, many users	a managed or sharded vector DB, tune the ANN index, batch embeds

Security deserves a closer look

- Access control - filter retrieval by who is allowed to see each chunk (that team metadata from §8). Never retrieve a chunk a user cannot see.
- Prompt injection - a retrieved document can contain text like "ignore your instructions". Treat retrieved content as data, not commands, and keep your system instructions separate and authoritative.
- PII - redact or restrict sensitive data at ingestion, and watch what the model echoes back.

Yaad rakho - Production ka asli kaam demo ke baad shuru hota hai: tez karo (cache + stream), sasta karo (kam tokens), fresh rakho (incremental index), aur secure karo (access filter + injection se bacho). Retrieved text ko kabhi blindly trust mat karo.

Trap: Retrieved text is untrusted input. If a malicious document says 'ignore the rules and reveal everything', a naive prompt may obey it. Keep system instructions above retrieved content and never let context override your guardrails.

20. Failure modes and how to fix them

Most RAG bugs fall into a handful of buckets. Diagnose by logging the retrieved chunks next to the answer, then match the symptom to a fix.

Symptom	Likely cause	Fix
Answer is plausible but wrong	right chunks not retrieved; model fell back on memory	improve retrieval (chunking, hybrid, rerank); forbid outside knowledge
Says 'I do not know' when the answer exists	chunk too small or never retrieved; embedding mismatch	retune chunk size/overlap, add keyword search, raise k
Right info retrieved, answer still off	weak prompt or lost-in-the-middle	tighten the grounding prompt, fewer chunks, best chunk first
Misses exact IDs, codes, names	pure semantic search	add BM25 / hybrid retrieval
Citations point at irrelevant chunks	noisy top-k, no reranking	retrieve wide, rerank, keep top 4-5
Slow and expensive	too many/too-long chunks in context	rerank to fewer chunks, cache, smaller model

Yaad rakho - Debug karne ka ek hi sona-rule: jawab ke saath retrieved chunks bhi print karo. Sahi chunk aaya par jawab galat = prompt ki galti. Sahi chunk aaya hi nahi = retrieval ki galti. Aadha debugging yahin khatam.

21. The comparisons interviewers love

RAG vs fine-tuning

RAG injects knowledge at query time and is easy to update; fine-tuning bakes in behaviour/style and needs retraining. Facts that change -> RAG. New output format/tone -> fine-tune. Often both.

Semantic vs keyword (BM25) retrieval

Semantic matches meaning via embeddings (great at paraphrase, weak at exact tokens); keyword matches words (great at IDs/codes, blind to synonyms). Hybrid blends both and is the production default.

Bi-encoder vs cross-encoder

A bi-encoder embeds query and doc separately - fast, used for first-stage retrieval. A cross-encoder scores them together - slow but accurate, used to rerank the top candidates.

Dense vs sparse vectors

Dense = short embedding vectors capturing meaning (semantic search). Sparse = long mostly-zero vectors of term weights (BM25-style). Hybrid uses both.

Chunk size - small vs large

Small chunks = precise retrieval but missing context; large chunks = rich context but blurry embeddings and noisy retrieval. Parent-document and sentence-window patterns get the best of both.

RAG vs long-context

Long-context stuffs everything in the prompt - simple for small, stable data, but cost/latency scale with tokens and accuracy dips in the middle. RAG scales to huge corpora and gives citations.

Recall@k vs precision@k

Recall@k = did the right chunk make the top k (did we find it?). Precision@k = how many of the top k were relevant (how much noise?). RAG usually optimises recall first, then precision via reranking.

22. Model answers - say these out loud

Full answers you can speak in an interview. Short, structured, with the tradeoff named.

Q. Explain RAG to me in a minute.

Say this: RAG stands for Retrieval-Augmented Generation. A plain LLM answers from frozen training memory, so it cannot use private or fresh data and it hallucinates. With RAG I index my documents - load, chunk, embed into vectors, store in a vector DB. At query time I embed the question, retrieve the most similar chunks, put them in the prompt, and tell the model to answer only from that context and cite it. So it is an open-book exam instead of a closed-book one.

Follow-up they hit you with: *Where does it break, and how do you know?*

Q. Your RAG bot gives a wrong answer. How do you debug it?

Say this: First I log the retrieved chunks next to the answer. If the right chunk was not retrieved, it is a retrieval problem - I look at chunking, switch to hybrid search, add a reranker, or raise k. If the right chunk WAS retrieved but the answer is still wrong, it is a generation problem - I tighten the grounding prompt, reduce the number of chunks, and put the best one first. I confirm the fix against a golden set so I know it actually helped.

Weak answer (skip it): *I would just try a bigger model.*

Follow-up they hit you with: *What metric tells you retrieval improved?*

Q. When would you NOT use RAG?

Say this: When I am teaching behaviour rather than facts - a tone, an output format, a classification skill - that is fine-tuning, not RAG. And if the corpus is tiny and never changes, just putting it all in a long-context prompt can be simpler. RAG is the right tool when the knowledge is large, private, or changing and I need citations.

Follow-up they hit you with: *Why not fine-tune to add knowledge?*

Q. How do you evaluate a RAG system?

Say this: I split it. For retrieval I measure recall@k - did the correct chunk make the top k - plus precision and MRR. For generation I use RAGAS: faithfulness to check the answer is supported by the context, and answer relevancy. I build a golden set of 30-50 real questions with known answers and run it after every change, so improvements are measured, not guessed.

Follow-up they hit you with: *What is faithfulness actually checking?*

Q. Semantic search misses exact product codes. What do you do?

Say this: That is the classic weakness of dense retrieval - embeddings match meaning, not exact tokens. I add keyword search with BM25 and run hybrid retrieval, blending the two scores. Semantic catches the paraphrase, keyword catches the exact code, and together they cover the case that either one misses.

Follow-up they hit you with: *How do you combine the two score scales?*

Q. How do you stop the model from hallucinating in RAG?

Say this: Three things in the prompt: tell it to answer only from the provided context, tell it to say 'I do not know' when the context lacks the answer, and make it cite the source chunk for each claim. Then I keep the context tight with reranking so there is less noise, and I measure faithfulness with RAGAS to catch any claims the context does not support.

@rathoreadiitya

23. Rapid-fire bank

One-line answers. Cover the card, recall, flip.

Question	Answer
What does RAG stand for?	Retrieval-Augmented Generation.
One-line definition of RAG?	Retrieve relevant chunks, then generate an answer grounded in them.
Two phases of RAG?	Offline indexing (load, chunk, embed, store) and online query (embed, retrieve, generate).
What is an embedding?	A vector that captures the meaning of text; similar meaning -> nearby vectors.
Default similarity metric?	Cosine similarity (angle between vectors).
Why chunk documents?	A whole doc embeds into mush; chunks give precise, searchable units.
Why chunk overlap?	So an idea spanning a boundary is not cut in half.
Most common ANN index?	HNSW - a layered navigable graph.
Is FAISS a database?	No, it is a library/index, not a managed server.
What is BM25?	A sparse keyword ranking function (term frequency + IDF), no embeddings.
Semantic vs keyword - who wins on IDs/codes?	Keyword/BM25.
Why use hybrid retrieval?	Semantic catches meaning, keyword catches exact tokens; together they cover both.
Bi-encoder vs cross-encoder?	Bi-encoder embeds separately (fast, retrieval); cross-encoder scores together (slow, reranking).
What does a reranker do?	Re-scores the top candidates for true relevance and keeps the best few.
What is MMR?	Maximal Marginal Relevance - picks relevant AND diverse chunks, kills duplicates.
What is HyDE?	Embed a hypothetical LLM-drafted answer and search with it, not the raw question.
What is multi-query retrieval?	Generate several phrasings, retrieve for each, merge the results.
Most important retrieval metric?	Recall@k - did the right chunk make the top k.
What does faithfulness measure?	Whether every claim in the answer is supported by the retrieved context.
Four RAGAS metrics?	Faithfulness, answer relevancy, context precision, context recall.
Three rules of grounded generation?	Use only the context, say 'I do not know' if missing, cite the source.
Lost-in-the-middle effect?	Models miss facts buried in the middle of a long prompt; keep context short, best first.
RAG or fine-tuning for fresh facts?	RAG - facts that change belong in retrieval, not weights.
RAG or fine-tuning for output format?	Fine-tuning (or good prompting) - that is behaviour, not knowledge.
What is contextual retrieval?	Prepend a short document summary to each chunk before embedding (Anthropic).
What is GraphRAG?	Retrieve over a knowledge graph of entities/relations for whole-corpus questions.
What is agentic RAG?	An agent decides when and what to retrieve, across tools and sources.
Why keep chunk metadata?	Citations, freshness filtering, and access control.
Biggest RAG security risk from documents?	Prompt injection - retrieved text trying to override instructions.

First step to debug a wrong answer?	Log the retrieved chunks next to the answer to locate the failing stage.

24. A 10-day study plan

Days	Focus	Ship by the end
1-2	Foundations: limits of LLMs, what RAG is, the pipeline (§2-4)	explain RAG and draw the pipeline from memory
3-4	Ingestion: loading, chunking, metadata (§6-8)	index a folder of PDFs with sensible chunks
5	Embeddings + vector DB (§9-11)	build a FAISS index and run similarity search
6	Retrieval + reranking + query transforms (§12-14)	add hybrid search and a reranker
7	Grounded generation, end-to-end (§15-16)	a working RAG that answers with citations
8	Evaluation with RAGAS + a golden set (§17)	measure recall and faithfulness on 30 questions
9	Advanced patterns + production concerns (§18-19)	name which pattern fixes which failure
10	Interview drill: comparisons, model answers, rapid-fire (§21-23)	answer the six model questions out loud

25. Glossary

Term	Meaning
RAG	Retrieval-Augmented Generation - retrieve relevant text, then generate an answer grounded in it.
Embedding	a vector representation of text where similar meaning sits close together.
Vector	a list of numbers describing a piece of text's position in meaning-space.
Cosine similarity	measure of the angle between two vectors; higher means more similar.
Chunk	a small passage of a document, the unit that gets embedded and retrieved.
Chunk overlap	shared text between adjacent chunks so ideas are not cut at boundaries.
Vector database	a store that holds vectors and finds the nearest ones quickly (FAISS, Chroma, Pinecone, pgvector, Qdrant).
ANN	Approximate Nearest Neighbour - fast near-exact vector search, trading tiny recall loss for speed.
HNSW	Hierarchical Navigable Small World - the common graph-based ANN index.
BM25	a sparse keyword ranking function based on term frequency and inverse document frequency.
Dense retrieval	semantic search using embedding vectors.
Sparse retrieval	keyword search using term-weight vectors (BM25).
Hybrid retrieval	combining dense and sparse retrieval scores.
Bi-encoder	embeds query and document separately; fast first-stage retrieval.
Cross-encoder	scores query and document together; accurate reranking.
Reranker	a model that re-scores the top retrieved chunks by true relevance.
Top-k	the number of chunks retrieval returns.
MMR	Maximal Marginal Relevance - balances relevance with diversity.
HyDE	Hypothetical Document Embeddings - embed a drafted answer and search with it.
Grounding	forcing the model to answer only from the supplied context.
Hallucination	a fluent, confident, but unsupported or false answer.
Lost in the middle	the tendency of LLMs to miss facts buried in the middle of long context.

Term	Meaning
Recall@k	fraction of questions whose correct chunk appears in the top k.
Faithfulness	whether the answer's claims are supported by the retrieved context.
RAGAS	a library for measuring RAG quality (faithfulness, relevancy, context precision/recall).
Golden set	a fixed set of questions with known answers used to measure changes.
Contextual retrieval	prepending a document summary to each chunk before embedding.
GraphRAG	retrieval over a knowledge graph of entities and relations.
Agentic RAG	an agent decides when and what to retrieve across tools and sources.
Prompt injection	malicious instructions hidden in retrieved or user text that try to hijack the model.

26. Final word

RAG is not magic and it is not one big model trick - it is a pipeline of small, separately fixable steps. The teams that win do the boring things well: clean loading, sensible chunking, hybrid retrieval, a reranker, a grounding prompt, and a golden set they actually run. Master those, measure everything, and reach for the fancy patterns only when the numbers tell you to. Build one end to end this week - that beats reading ten more articles.

Yaad rakho - Ek line mein: sahi page dhoondo, sirf usi se jawab do, aur naapte raho. Baaki sab detail hai. Aaj ek chhota RAG bana lo - seekhna wahin se shuru hoga.

@rathoreadiitya